

RAPPORT DE PROJET BUSINESS INTELLIGENCE

Conception et implémentation d'un Data Lake sur AWS pour le traitement, l'analyse et la visualisation des données Scopus à l'aide de Power BI

réalisée par	ACHRAF EL BOUMASHOULI AFAILAL TREBAK GHIZLANE
Encadrant	Pr. M. SALMA CHRIT
Module	Business Intelligence
Annee	2025 / 2026

Résumé

Ce projet porte sur la conception et la mise en œuvre d'une architecture Data Lake sur Amazon Web Services (AWS) pour le traitement et l'analyse de données bibliographiques issues de la plateforme Scopus. L'objectif est de centraliser des données hétérogènes (structurées, semi-structurées, non structurées), de les transformer automatiquement via un pipeline ETL et de les rendre exploitables à travers des tableaux de bord interactifs.

Amazon S3 assure le stockage des données selon une architecture en trois zones (Bronze, Silver, Gold). AWS Lambda automatise l'ingestion et la décompression. AWS Glue orchestre les transformations ETL (PySpark). Amazon Athena permet l'interrogation SQL serverless. Power BI assure la visualisation finale. Des défis liés aux doublons et à la normalisation des affiliations ont été résolus par des fichiers de correspondance JSON.

Mots-clés : Data Lake, AWS, Amazon S3, AWS Glue, Athena, Scopus, bibliométrie, Power BI, pipeline ETL, universités marocaines.

Abstract

This project focuses on designing and implementing a Data Lake architecture on Amazon Web Services (AWS) to process and analyze bibliographic data from the Scopus platform. The goal is to centralize heterogeneous data, automatically transform it through an ETL pipeline, and make it exploitable through interactive dashboards.

Amazon S3 stores data in a three-layer architecture (Bronze, Silver, Gold). AWS Lambda automates ingestion and decompression. AWS Glue orchestrates ETL transformations using PySpark. Amazon Athena enables serverless SQL querying. Power BI provides the final visualization layer. Challenges related to duplicates and affiliation normalization were addressed through JSON mapping files.

Keywords: Data Lake, AWS, Amazon S3, AWS Glue, Athena, Scopus, bibliometric analysis, Power BI, data pipeline, Moroccan universities.

Table des abréviations

AWS	Amazon Web Services
S3	Simple Storage Service
ETL	Extract, Transform, Load
JSON	JavaScript Object Notation
CSV	Comma-Separated Values
SQL	Structured Query Language
IAM	Identity and Access Management
KMS	Key Management Service
UDF	User Defined Function
KPI	Key Performance Indicator
API	Application Programming Interface
ZIP	Format d'archive compressée

Introduction Générale

Le monde numérique génère des volumes exponentiels de données — réseaux sociaux, capteurs IoT, plateformes scientifiques — représentant potentiellement 175 zettaoctets selon l'IDC. Les bases de données relationnelles traditionnelles montrent leurs limites face à cette diversité : un fichier JSON de capteur IoT, un CSV scientifique et une image satellitaire requièrent des traitements fondamentalement différents.

La solution : le Data Lake. Contrairement aux entrepôts de données classiques qui imposent un schéma rigide à l'écriture (schema-on-write), le Data Lake adopte une approche schema-on-read : les données sont stockées brutes et la structure est appliquée uniquement à la consultation. Sur la plateforme AWS, les services S3, Glue, Lambda et Athena permettent de construire des architectures modernes, serverless, scalables et économiques.

Ce projet met en place un Data Lake AWS pour analyser les données bibliographiques de Scopus (publications scientifiques des universités marocaines), avec pour finalité des tableaux de bord interactifs sous Power BI. Le rapport s'articule en trois chapitres : fondations théoriques, conception et implémentation, mise en œuvre et analyse des résultats.

CHAPITRE 1 : Fondations Théoriques et Technologiques

1. Data Lake — Définition et avantages

Un Data Lake est un dépôt centralisé permettant de stocker tous types de données dans leur format brut, sans transformation préalable. Il repose sur trois principes : centralisation (source unique), évolutivité (capacité élastique) et accessibilité (compatible SQL, ML, visualisation). Ses avantages majeurs sont :

- Flexibilité : intégration de nouvelles sources sans redéfinir un schéma existant.
- Coût réduit : stockage cloud environ 0,023 USD/Go/mois (Amazon S3 Standard), sans infrastructure physique.
- Scalabilité : élasticité automatique face aux volumes Big Data imprévisibles.
- Accessibilité IA/Data Science : manipulation des données brutes avec Apache Spark, TensorFlow, AWS Glue, Athena.

Exemple réel : Netflix stocke plus de 100 pétaoctets de données vidéo, logs utilisateurs et retours clients dans un Data Lake sur AWS S3.

2. Justification du choix AWS

AWS s'impose par : (1) scalabilité automatique d'Amazon S3, (2) intégration native entre services (Glue, Athena, Lambda, QuickSight), (3) sécurité robuste (IAM, KMS, TLS), (4) modèle pay-as-you-go adapté aux projets académiques (AWS Educate, Free Tier), (5) documentation riche et communauté active. Hadoop, l'alternative open-source, nécessite des clusters physiques coûteux et une administration complexe.

3. Services AWS utilisés

Amazon S3	Stockage objets scalable (durabilité 99,999999999 %). Organisation en buckets (bronze/silver/gold). Supporte JSON, CSV, Parquet, ZIP, images.
AWS Lambda	Fonctions serverless déclenchées par événements S3. Décompression ZIP, nettoyage CSV, envoi silver.
AWS Glue	Moteur ETL serverless. Jobs PySpark pour extraction, transformation, chargement.
AWS Glue Crawler	Scan automatique S3, détection des schémas, mise à jour du Data Catalog.
AWS Glue Workflow	Orchestration des jobs ETL, crawlers et déclencheurs en chaîne séquentielle.
Amazon CloudWatch	Surveillance en temps réel, logs d'exécution, alertes automatiques.
Amazon Athena	Requêtes SQL serverless directement sur S3. Supporte Parquet, JSON, CSV.
Power BI	Business Intelligence Microsoft. Connexion à Athena via ODBC. Tableaux de bord interactifs.

CHAPITRE 2 : Conception et Implémentation

1. Objectifs techniques

Le projet vise à construire une plateforme AWS de traitement des données bibliographiques Scopus pour appuyer les décisions stratégiques en recherche et enseignement supérieur. Les objectifs principaux sont :

- Centralisation des données CSV bruts (format ZIP) dans Amazon S3.
- Structuration en zones Bronze / Silver / Gold selon le niveau de traitement.
- Automatisation complète : AWS Lambda pour l'ingestion, AWS Glue pour le pipeline ETL.
- Normalisation des affiliations via fichiers JSON (villes, universités, écoles marocaines).
- Modèle de données relationnel : auteurs, documents, journaux, affiliations.
- Interrogation SQL via Amazon Athena et visualisation via Power BI connecté en ODBC.

2. Présentation de la base Scopus

Scopus est la base bibliographique internationale d'Elsevier (lancée en 2004), indexant plus de 88 millions de documents issus de 25 000+ revues évaluées par les pairs. Elle couvre sciences naturelles, ingénierie, médecine, sciences sociales, arts. Ses métadonnées enrichies (auteurs, affiliations, DOI, citations, SJR, h-index, CiteScore) permettent des analyses bibliométriques avancées. Les Scopus Author IDs assurent la désambiguïsation des noms d'auteurs.

3. Description des données sources

Trois types de données sources ont été utilisés :

- scopus1913_2025.zip : archive compressée de données bibliographiques (1913–2025). Colonnes principales : Auteurs, Titre, Année, Source, DOI, Affiliations, Résumé, Mots-clés, Citations, Langue, Type de document, ISSN, EID. Traitement par Lambda → CSV séparateur |.
- Fichiers JSON (city.json, university.json, school.json) : référentiels de normalisation des affiliations marocaines, incluant variantes orthographiques et abrégées.
- scimagojr.csv : indicateurs bibliométriques des journaux (SJR, H-index, quartiles, total citations, éditeur, pays, ISSN).

	Authors	Author full names	Author(s) ID	Title	Year	Source title	Volume	Issue	Art. No.	Page start	...	ISBN	CODEN	PubMed ID	Language of Original Document	Abbreviated Source Title	Document Type	Publication Stage	Open Access	Source	EID
0	Nadif B.	Nadif, Bendaoud (59482605100)	59482605100	Unveiling the relationships between language L...	2025	Journal of Interdisciplinary Studies in Education	14	1.0	NaN	20	...	NaN	NaN	NaN	English	J. Interdiscip. Stud. Educ.	Article	Final	All Open Access; Hybrid Gold Open Access	Scopus	2-e2-0-85212778437
1	Zaki K.; Sbati A.; Bouachrine M.; Lakhilfi T.	Zaki, Khadija (58806762100); Sbati, Abdelouahid... (59248797500);	58806762100; 57194341829; 8983993500; 58082496100	Regioselectivity study of 1,3-dipolar cycloadd...	2025	Current Chemistry Letters	14	NaN	NaN	183	...	NaN	NaN	NaN	English	Curr. Chem. Lett.	Article	Final	NaN	Scopus	2-e2-0-85210081337
2	Ouachekradi M.; Elkabous M.; Karzazi Y.	Ouachekradi, Mohammed (59248797500); Elkabous... (6602399647)	59248797500; 6602399647	Triphenylamine-based D-A- π -A dyes for DSSC app...	2025	Journal of Photochemistry and Photobiology A...	461	NaN	116152	NaN	...	NaN	JPPCE	NaN	English	J. Photochem. Photobiol. A Chem.	Article	Final	NaN	Scopus	2-e2-0-85209240189
3	Tayiem A.N.; Fares O.; Laili B.A.; Hamed O.; De...	Tayiem, A.N. (59474863900); Fares, O. (57209233422); Fares, O. (57209233422); Hamed, O. (57209233422); De... (7003819...	59474863900; 57209233422; 57209233422; 57209233422; 7003819...	Cellulose functionalized with amino and mercap...	2025	Journal of Molecular Structure	1326	NaN	141099	NaN	...	NaN	JMOSB	NaN	English	J. Mol. Struct.	Article	Final	NaN	Scopus	2-e2-0-85212230504
4	Sebhaoui J.; Ashraf S.; Iqbal S.; Haili M.; Be...	Sebhaoui, Jihad (57192958440); Ashraf, Sajda (56957832700); Iqbal, Sajda (8701242900); Haili M.; Be... (56191077...	57192958440; 56957832700; 8701242900; 56191077...	Synthesis of novel tetrahydrobenzo(b)thiophene...	2025	Journal of Molecular Structure	1326	NaN	141110	NaN	...	NaN	JMOSB	NaN	English	J. Mol. Struct.	Article	Final	NaN	Scopus	2-e2-0-85212573920

Figure 1 : Données brutes extraites de Scopus — vue tabulaire des colonnes principales

4. Architecture cible sur AWS

L'architecture repose sur six composants intégrés en mode serverless :

Amazon S3	Stockage principal. Zones : bronze/ (données brutes ZIP/CSV), silver/ (données nettoyées, séparateur , UTF-8), gold/ (tables analytiques enrichies), ressources/ (JSON de référence).
AWS Lambda	Déclenchement automatique sur événement s3:ObjectCreated. Décompression ZIP, conversion CSV, nettoyage encodage, transfert vers silver.
AWS Glue ETL	4 jobs PySpark : AffiliationExtractionJob, AuthorExtractionJob, DocumentExtractionJob, JournalJob.
AWS Glue Crawler	Scan récursif de gold/, génération des tables dans le Glue Data Catalog.
Amazon Athena	Requêtes SQL sur les fichiers CSV gold/ via le Data Catalog.
Power BI	Connexion ODBC à Athena. 7 pages interactives de visualisation.

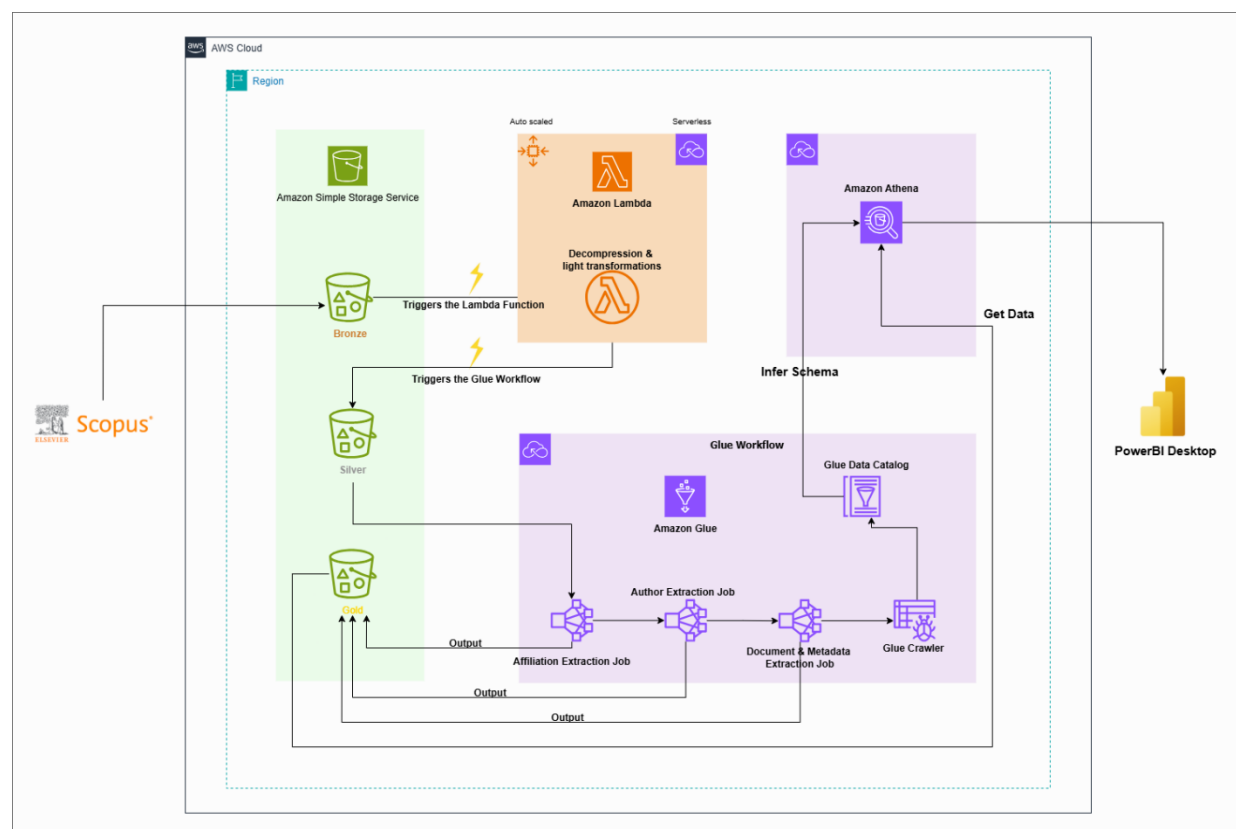


Figure 2 : Architecture du pipeline d'ingestion, de transformation et de visualisation — Vue AWS Cloud

L'architecture serverless et modulaire garantit scalabilité, tolérance aux pannes et gestion simplifiée, du stockage initial jusqu'à la visualisation finale, sans gestion d'infrastructure.

CHAPITRE 3 : Mise en Œuvre et Analyse des Résultats

1. Mise en œuvre technique

1.1. Structuration du stockage S3

Le bucket S3 (amazon-scopus-bucket) est organisé en quatre zones logiques conformes aux bonnes pratiques Data Lake cloud-native :

- bronze/ : données brutes immuables (scopus1913_2025.zip, scimagojr.csv). Garantit la reproductibilité des traitements.
- silver/ : données décompressées, nettoyées (séparateur | au lieu de virgule, encodage UTF-8, en-têtes standardisés). Produits par Lambda. 9 fichiers CSV générés (scopus1913to2007.csv, scopus2008-2013.csv, ... scopus2025.csv).
- gold/ : tables analytiques enrichies par Glue : AffiliationExtractionOutput/, AuthorExtractionOutput/, DocumentExtractionOutput/, JournalOutput/, MetadataExtractionOutput/.
- resources/ : référentiels JSON — city.json (8,6 KB), school.json (30,9 KB), university.json (18,9 KB).

Cette séparation permet : gestion indépendante des couches, permissions spécifiques, politiques de cycle de vie différenciées, scalabilité horizontale.

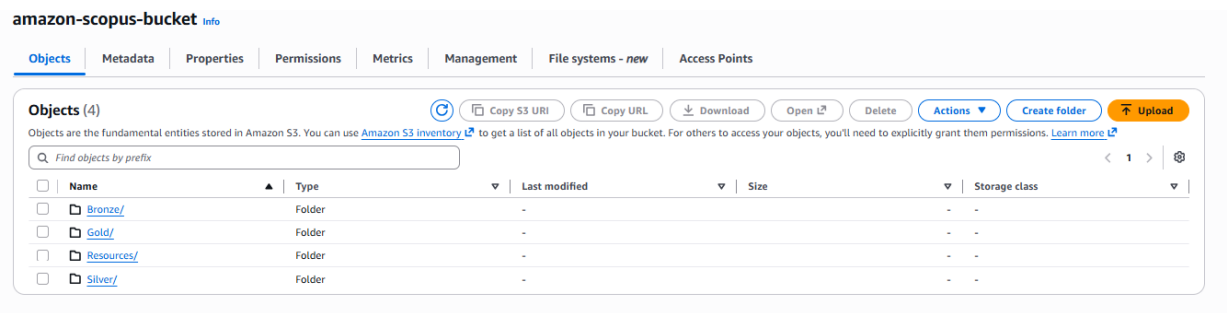


Figure 3 : Structure hiérarchique des dossiers dans le bucket S3

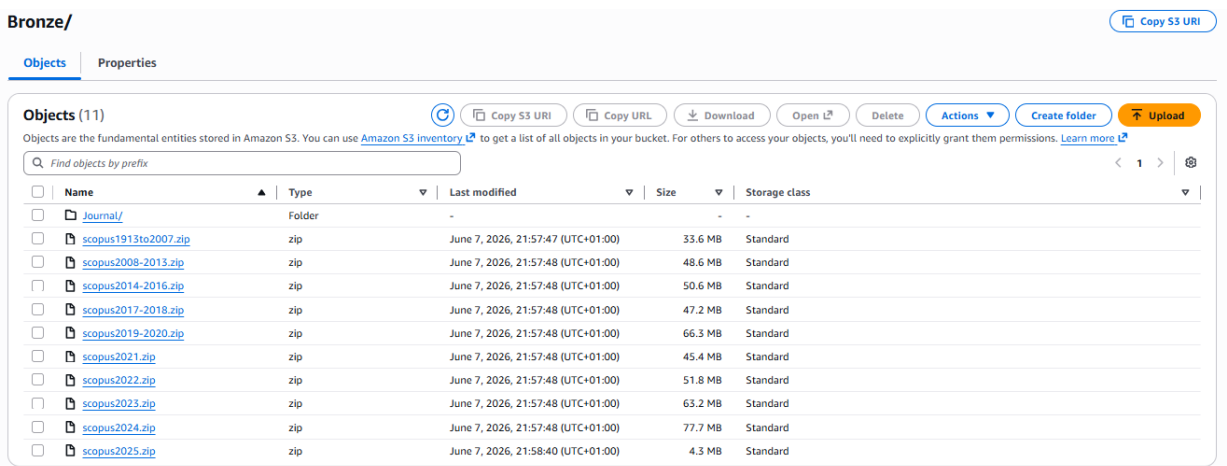


Figure 4 : Contenu du niveau Bronze (données brutes compressées)

Silver/ Copy S3 URI

Objects Properties

Objects (10) Copy S3 URI Copy URL Download Open Delete Actions Create folder Upload

Find objects by prefix

☐	Name	Type	Last modified	Size	Storage class
☐	scopus1913to2007.csv	csv	June 8, 2026, 00:37:52 (UTC+01:00)	94.5 MB	Standard
☐	scopus2008-2013.csv	csv	June 8, 2026, 00:36:29 (UTC+01:00)	142.2 MB	Standard
☐	scopus2014-2016.csv	csv	June 8, 2026, 00:35:52 (UTC+01:00)	149.4 MB	Standard
☐	scopus2017-2018.csv	csv	June 8, 2026, 00:28:52 (UTC+01:00)	138.7 MB	Standard
☐	scopus2019-2020.csv	csv	June 8, 2026, 00:26:39 (UTC+01:00)	192.3 MB	Standard
☐	scopus2021.csv	csv	June 8, 2026, 00:25:03 (UTC+01:00)	129.6 MB	Standard
☐	scopus2022.csv	csv	June 8, 2026, 00:24:44 (UTC+01:00)	148.2 MB	Standard
☐	scopus2023.csv	csv	June 8, 2026, 00:23:57 (UTC+01:00)	181.0 MB	Standard
☐	scopus2024.csv	csv	June 8, 2026, 00:22:20 (UTC+01:00)	222.9 MB	Standard
☐	scopus2025.csv	csv	June 8, 2026, 00:08:17 (UTC+01:00)	12.3 MB	Standard

Figure 5 : Couche Silver — fichiers CSV nettoyés (scopus1913to2007.csv à scopus2025.csv)

Gold/ Copy S3 URI

Objects Properties

Objects (5) Copy S3 URI Copy URL Download Open Delete Actions Create folder Upload

Find objects by prefix

☐	Name	Type	Last modified	Size	Storage class
☐	AffiliationExtractionOutput/	Folder	-	-	-
☐	AuthorExtractionOutput/	Folder	-	-	-
☐	DocumentExtractionOutput/	Folder	-	-	-
☐	Journal Output/	Folder	-	-	-
☐	MetadataExtractionOutput/	Folder	-	-	-

Figure 6 : Couche Gold — dossiers d'extraction des 4 jobs Glue

Resources/ Copy S3 URI

Objects Properties

Objects (3) Copy S3 URI Copy URL Download Open Delete Actions Create folder Upload

Find objects by prefix

☐	Name	Type	Last modified	Size	Storage class
☐	city.json	json	May 28, 2026, 02:39:40 (UTC+01:00)	8.6 KB	Standard
☐	school.json	json	May 28, 2026, 02:39:40 (UTC+01:00)	30.9 KB	Standard
☐	university.json	json	May 28, 2026, 02:39:41 (UTC+01:00)	18.9 KB	Standard

Figure 7 : Répertoire Resources — fichiers JSON de référence

1.2. Fonction AWS Lambda

La fonction Lambda (bronze_to_silver_etl.py) se déclenche sur chaque événement s3:ObjectCreated dans le dossier bronze/. Son rôle : identifier les fichiers .zip, les télécharger en mémoire via l'API get_object, décompresser avec la bibliothèque Python zipfile, parcourir les fichiers CSV contenus, puis pour chaque CSV :

- Corriger le séparateur (virgule → |).
- Normaliser l'encodage (UTF-8).
- Renommer les en-têtes non conformes (ex. "Art. No." → "Art_no").
- Gérer les caractères d'échappement.
- Sauvegarder le fichier nettoyé dans silver/ via put_object.

Les logs CloudWatch permettent la surveillance en continu de chaque exécution (Reading CSV file... / Processed file saved to...). Les avantages : réactivité immédiate, scalabilité dynamique, coût minimal (facturation à l'usage), aucune infrastructure dédiée.

Lambda-function

▼ Function overview [Info](#)

Diagram | Template



Lambda-function



Layers

(0)



S3

(2)

+ Add trigger

+ Add destination

Trigger



S3: [amazon-scopus-bucket](#)

arn:aws:s3::amazon-scopus-bucket

▼ Details

Bucket arn: [arn:aws:s3::amazon-scopus-bucket](#)

Event types: [s3:ObjectCreated:Put](#), [s3:ObjectCreated:CompleteMultipartUpload](#)

isComplexStatement: **No**

Notification name: [0fce0ff-c262-41ad-b85a-6981db7a4555](#)

Prefix: **Bronze/**

Service principal: [s3.amazonaws.com](#)

Source account: [207785581053](#)

Statement ID: [lambda-7d87f412-444d-4ac9-a4be-125360da2944](#)

Suffix: **.zip**

Figure 8 : Déclencheur AWS Lambda

```
bronze_to_silver_etl.py
9 def lambda_handler(event, context):
10     try:
11         bucket_name = event['bucket_name']
12         file_key = event['file_key']
13         destination_folder = event['destination_folder']
14
15         print(f"Processing file: {file_key}")
16
17         # Check if the file is a .zip file
18         if file_key.endswith('.zip'):
19             # Download the zip file
20             zip_file = s3.get_object(Bucket=bucket_name, Key=file_key)['Body'].read()
21
22             # Open the zip file
23             with zipfile.ZipFile(io.BytesIO(zip_file), 'r') as zip_ref:
24                 # Extract all files in the zip archive
25                 for file_info in zip_ref.infolist():
26                     # Check if the file is a CSV file
27                     if file_info.filename.endswith('.csv'):
28                         print(f'Reading CSV file: {file_info.filename}')
29
30                         # Read the CSV file from the zip archive
31                         with zip_ref.open(file_info) as file:
32                             # Use io.TextIOWrapper to decode the file
33                             csv_file = io.TextIOWrapper(file, encoding='utf-8')
34
35                             # Read the CSV file line by line
36                             reader = csv.reader(csv_file)
37                             header = next(reader) # Read the header
38
39                             # Prepare the output with the new delimiter (|)
```

Figure 9 : Script Lambda — décompression des fichiers ZIP avec zipfile.ZipFile

```

9 def lambda_handler(event, context):
10     try:
11         if file_key.endswith('.zip'):
12             with zipfile.ZipFile(io.BytesIO(zip_file), 'r') as zip_ref:
13                 for file_info in zip_ref.infolist():
14                     # Read the CSV file from the zip archive
15                     with zip_ref.open(file_info) as file:
16                         # Use io.TextIOWrapper to decode the file
17                         csv_file = io.TextIOWrapper(file, encoding='utf-8')
18
19                     # Read the CSV file line by line
20                     reader = csv.reader(csv_file)
21                     header = next(reader) # Read the header
22
23                     # Prepare the output with the new delimiter (|)
24                     output = io.StringIO()
25                     writer = csv.writer(output, delimiter='|', quotechar=None, quoting=csv.QUOTE_NONE, escapechar='\\')
26
27                     # Rename the column "Art. No." to "Art_no" in the header
28                     if "Art. No." in header:
29                         header[header.index("Art. No.")] = "Art_no"
30
31                     # Write the updated header
32                     writer.writerow(header)
33
34                     # Process each row
35                     for row in reader:
36                         writer.writerow(row)
37
38                     # Define the output file key
39                     output_file_key = f"{destination_folder}/{file_info.filename}"
40
41                     # Upload the updated CSV file to S3
42                     s3.put_object(Bucket=bucket_name, Key=output_file_key, Body=output.getvalue())
43                     print(f'Processed file saved to: {output_file_key}')
44
45

```

Figure 10 : Script Lambda — conversion CSV, remplacement séparateur, normalisation UTF-8 et traités vers S3 via put_object

▼	Timestamp	Message
▶	2026-06-09T09:23:59.066Z	Reading CSV file: scopus1913to2007.csv
▶	2026-06-09T09:23:59.611Z	Processed file saved to: 2-silver/scopus1913to2007.csv
▶	2026-06-09T09:23:59.611Z	Reading CSV file: scopus2008-2013.csv
▶	2026-06-09T09:23:59.611Z	Processed file saved to: 2-silver/scopus2008-2013.csv
▶	2026-06-09T09:23:59.704Z	Reading CSV file: scopus2014-2016.csv
▶	2026-06-09T09:24:00.429Z	Processed file saved to: 2-silver/scopus2014-2016.csv
▶	2026-06-09T09:24:00.431Z	Reading CSV file: scopus2017-2018.csv
▶	2026-06-09T09:24:00.738Z	Processed file saved to: 2-silver/scopus2017-2018.csv
▶	2026-06-09T09:24:00.741Z	Reading CSV file: scopus2019-2020.csv
▶	2026-06-09T09:24:00.741Z	Processed file saved to: 2-silver/scopus2019-2020.csv
▶	2026-06-09T09:24:52.368Z	Reading CSV file: scopus2021.csv
▶	2026-06-09T09:24:52.369Z	Processed file saved to: 2-silver/scopus2021.csv
▶	2026-06-09T09:24:52.369Z	Reading CSV file: scopus2022.csv
▶	2026-06-09T09:24:52.369Z	Processed file saved to: 2-silver/scopus2022.csv
▶	2026-06-09T09:24:52.882Z	Reading CSV file: scopus2023.csv
▶	2026-06-09T09:25:00.215Z	Processed file saved to: 2-silver/scopus2023.csv
▶	2026-06-09T09:25:00.228Z	Reading CSV file: scopus2024.csv
▶	2026-06-09T09:25:00.444Z	Processed file saved to: 2-silver/scopus2024.csv
▶	2026-06-09T09:25:00.447Z	Reading CSV file: scopus2025.csv
▶	2026-06-09T09:25:00.447Z	Processed file saved to: 2-silver/scopus2025.csv
▶	2026-06-09T09:25:25.683Z	Decompression completed successfully: "File processed successfully!"

Figure 11 : Journaux CloudWatch — état de traitement des 9 fichiers CSV de l'archive Scopus

1.3. Pipeline ETL avec AWS Glue

Quatre jobs Glue (version 5.0, PySpark) transforment les données silver en tables analytiques gold :

AffiliationExtractionJob	Extraction et normalisation des affiliations. Utilise city.json, university.json, school.json via UDF PySpark avec expressions régulières. Produit la table des affiliations avec identifiants uniques (affiliation_id, ville, université, école, pays, laboratoire).
AuthorExtractionJob	Extraction des auteurs depuis la colonne author_full_names (format : Nom, Prénom (ID)). Génère la table des auteurs (author_id, author_name) et la table de relation auteurs-documents.
DocumentExtractionJob	Nettoyage des métadonnées scientifiques. Extrait DOI, titre, abstract, mots-clés, langue, type, références. Suppression des doublons via dropDuplicates(["doc_id"]). Normalisation des EID (suppression du préfixe "2-s2.0-"). Produit document_table et metadata_table.

JournalJob

Traitement de scimagojr.csv : suppression caractères spéciaux (|), conversion numérique (virgule → point pour SJR, %Female), typage (H index → IntegerType), déduplication sur ISSN. Produit journal_table enrichi (SJR, H-index, quartiles, publications).

Workflows (2)			
A workflow is an orchestration used to visualize and manage the relationship and execution of multiple triggers, jobs and crawlers.			
Filter workflows			
☐	Name	Last run	Last run status
☐	scopus_workflow	Mon, 08 Jun 2026 00:13:17 GMT	Completed

Figure 12 : Interface AWS Glue — 4 jobs ETL

1.4. Orchestration avec AWS Glue Workflow

Le workflow workflow_scopus chaîne l'ensemble des traitements de manière entièrement automatisée. Séquence d'exécution :

- Déclencheur initial (First_OnDemand_Trigger) → AffiliationExtractionJob
- Author_Trigger → AuthorExtractionJob
- Document_Job_Trigger → DocumentExtractionJob
- Journal_Job_Trigger → JournalTransformationJob
- Crawler_Trigger (ALL) → Crawler_Scopus

Les dépendances séquentielles garantissent l'ordre fonctionnel (affiliations avant auteurs). Le crawler final rafraîchit automatiquement le Data Catalog. En cas d'échec, le graphe dynamique dans la console Glue identifie immédiatement le nœud bloquant pour relance ciblée.

Workflow details			
Advanced properties			
Name	Description	Max concurrency	Last run status
scopus_workflow	-	-	Completed
Last run	Last modified	Blueprint name	Blueprint run id
June 8, 2026 at 00:13:17	May 27, 2026 at 20:49:29	-	-

Figure 13 : Détails du workflow AWS Glue workflow_scopus — Last run status: Completed

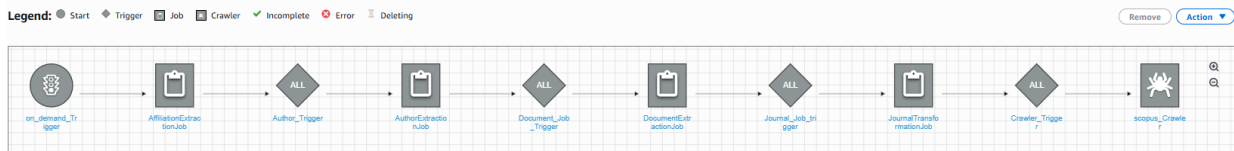


Figure 14 : Graphe du workflow Glue — chaîne séquentielle des jobs et déclencheurs

1.5. Catalogue et interrogation (Glue Crawler & Athena)

Le Crawler_Scopus pointe vers s3://amazon-scopus-bucket/gold/ en scan récursif. Il détecte les fichiers CSV (séparateur |) et génère dans la base de données scopus du Glue Data Catalog 7 tables : affiliation_table, author_table, documentextractionoutput, intermediate_affiliation_metadata, intermediate_author_metadata, journal_output, metadataextractionoutput.

Amazon Athena se connecte automatiquement au Data Catalog. Toute nouvelle table détectée par le crawler devient immédiatement interrogeable en SQL, sans écrire de CREATE TABLE. Exemples de requêtes possibles : nombre d'articles par université, auteurs les plus productifs, publications par année, classement des journaux par SJR.

Review and update

Step 1: Set crawler properties

Set crawler properties

Name

scopus_crawler

Description

-

Tags

-

Step 2: Choose data sources and classifiers

Data sources (1)

The list of data sources to be scanned by the crawler.

Type

S3

Data source

s3://amazon-scopus-bucket/Gold/

Parameters

Recrawl all

Step 3: Configure security settings

Configure security settings

IAM role

AssumeRole

Security configuration

-

Lake Formation configuration

-

Step 4: Set output and scheduling

Set output and scheduling

Database

scopus-database

Table prefix - optional

-

Maximum table threshold - optional

-

Schedule

On demand

Cancel

Previous

Update

Figure 15: Configuration du Crawler_Scopus

scopus-database

Last updated (UTC)

June 10, 2026 at 18:42:12

Edit

Delete

Database properties

Name

scopus-database

Description

-

Location

-

Created on (UTC)

May 27, 2026 at 20:59:24

Tables (8)

View and manage all available tables.

Last updated (UTC)

June 10, 2026 at 18:42:08

Delete

Add tables using crawler

Add table

Name	Database	Location	Classification	Deprecated	View data	Data quality
affiliation_table	scopus-database	s3://amazon-scopus-bucket/Gold/Affiliat	CSV	-	Table data	View data quality
author_table	scopus-database	s3://amazon-scopus-bucket/Gold/Author	CSV	-	Table data	View data quality
documentextractionoutput	scopus-database	s3://amazon-scopus-bucket/Gold/Docum	CSV	-	Table data	View data quality
intermediate_affiliation_metadata	scopus-database	s3://amazon-scopus-bucket/Gold/Affiliat	CSV	-	Table data	View data quality
intermediate_author_metadata	scopus-database	s3://amazon-scopus-bucket/Gold/Author	CSV	-	Table data	View data quality
journal_output	scopus-database	s3://amazon-scopus-bucket/Gold/Journa	CSV	-	Table data	View data quality
metadata_table	scopus-database	s3://amazon-scopus-bucket/Gold/Affiliat	CSV	-	Table data	View data quality
metadataextractionoutput	scopus-database	s3://amazon-scopus-bucket/Gold/Metad	CSV	-	Table data	View data quality

Figure 16 : Glue Data Catalog — 7 tables générées dans la zone Gold

Editor

Recent queries

Saved queries

Query settings

Workgroup

primary

Data

Data source

AwsDataCatalog

Catalog

None

Database

scopus-database

Tables and views

Create

Filter tables and views

Tables (8)

affiliation_table

author_table

documentextractionoutput

intermediate_affiliation_metadata

intermediate_author_metadata

journal_output

metadata_table

metadataextractionoutput

Views (0)

Query 1

Query 2

select * from affiliation_table;

Run again

Explain

Cancel

Clear

Create

Query results

Query stats

Completed

Time in queue: 101 ms

Run time: 834 ms

Data scanned: 10.65 MB

Results (38,850)

Search rows

Copy

Download results CSV

#	join_key	aff_clean
1	06602eb4802d4ad5f9b79e712e92d7b42e56ee7b611391209fde84046543d	dit internationale university of rabat morocco
2	fb10d9fa4565404477b6538577008b4626c180a949d54208677b17b4e784	capm centre arti poison et de pharmacovigilance du maroc a rabat morocco
3	acd757c8a549e37f6e532a33953f555b9e291150fb446b7a130f46beba6db25	creheho centre de recherche de lecole des hautes etudes dingerierie oujda morocco
4	e54u894f8b2a24f005345b2280c0b0f9634644438c9ff78bf0be2c72317a7c	iranotech institute of nanomaterials and nanotechnology masir moroccan foundation for advanced science innovation and research enset madinat el ifrane 10100 rabat avenue de l'armee royale me
5	55e5d678bde436e9e47c8f9c2f7a8171604b948878996fa8b999a54d6d3275	iranotech institute of nanomaterials and nanotechnology masir moroccan advanced science innovation and research foundation enset rabat av armee royale morocco

Figure 17 : Interface Amazon Athena — éditeur de requêtes, base scopus, liste des tables

Results (26)

Copy

Download results CSV

< 1 > @

#	▼	university	▼	nombre_articles	▼
1		Mohammed V University		5285	
2		Hassan II University		4822	
3		Cadi Ayyad University		3434	
4		Abdelmalek Essaâdi University		3421	
5		Sidi Mohamed Ben Abdellah University		2854	
6		Ibn Tofail University		2373	
7		University Mohammed I		1858	
8		Moulay Ismail University		1714	
9		University Sultan Moulay Slimane		1323	
10		Chouaib Doukkali University		1245	
11		Ibn Zohr University		1241	
12		International University of Casablanca		192	
13		National Institute of Posts and Telecommunications of Rabat		182	
14		Mohammed VI Polytechnic University		114	
15		Agronomic and Veterinary Institute Hassan II of Rabat		39	
16		Euro-Mediterranean University of Morocco		34	
17		International University of Rabat		28	
18		Mohammed VI University of Health Sciences of Casablanca		25	

Figure 18: Résultat de requête SQL Athena — nombre d'articles par université (Mohammed V: 5285, Hassan II: 4822, Cadi Ayyad: 3434 ...)

1.6. Visualisation interactive avec Power BI

Power BI est connecté à Athena via ODBC. Les tables `author_table`, `document_table`, `affiliation_table` et `journal_table` sont liées par relations logiques (clé primaire / clé étrangère). Le rapport final contient 7 pages interactives :

Page 1	Vue géographique et institutionnelle : carte interactive, KPIs (39K affiliations, 138K articles, 235K auteurs), diagramme Sankey écoles-types de publications. Filtres : école, université, ville, période.
Page 2	Contribution des laboratoires et universités : barres horizontales (labs), diagramme circulaire (universités). Identification des acteurs scientifiques majeurs.
Page 3	Titres et types de publications : liste articles récents, barres par type (journal 68K, book series 5K, conference 4K), courbe temporelle par catégorie.
Page 4	Articles par année et par université : progression annuelle 1913–2025, tableau comparatif des 18 universités marocaines (Total: 37 483 articles filtrés).
Page 5	Auteurs, types de journaux, éditeurs : tableau auteurs-articles-type-année (142 654 total), répartition journaux (journal 100%), barres éditeurs.
Page 6	Types de documents et affiliations : articles 6,98K → reviews 20,13% → conference papers 5,84%... Diagramme circulaire affiliations par université. Table éditeur-revue.
Page 7	Mots-clés et langues : nuage de mots-clés (Optimization, Corrosion, DFT, Spectroscopy...), répartition langues (anglais 90,58%, français, espagnol, arabe...).

Figure 19 : Vue géographique et institutionnelle — carte Maroc + Sankey écoles-publications

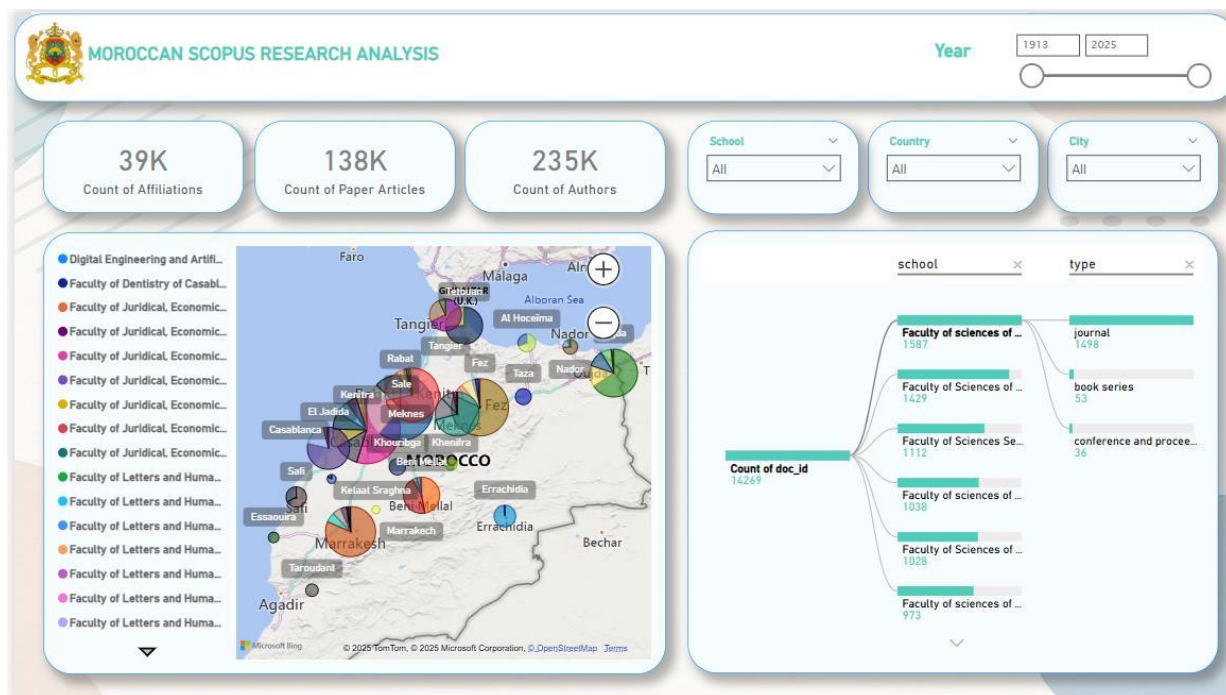


Figure 20 : Contribution des laboratoires et universités marocaines à la recherche scientifique

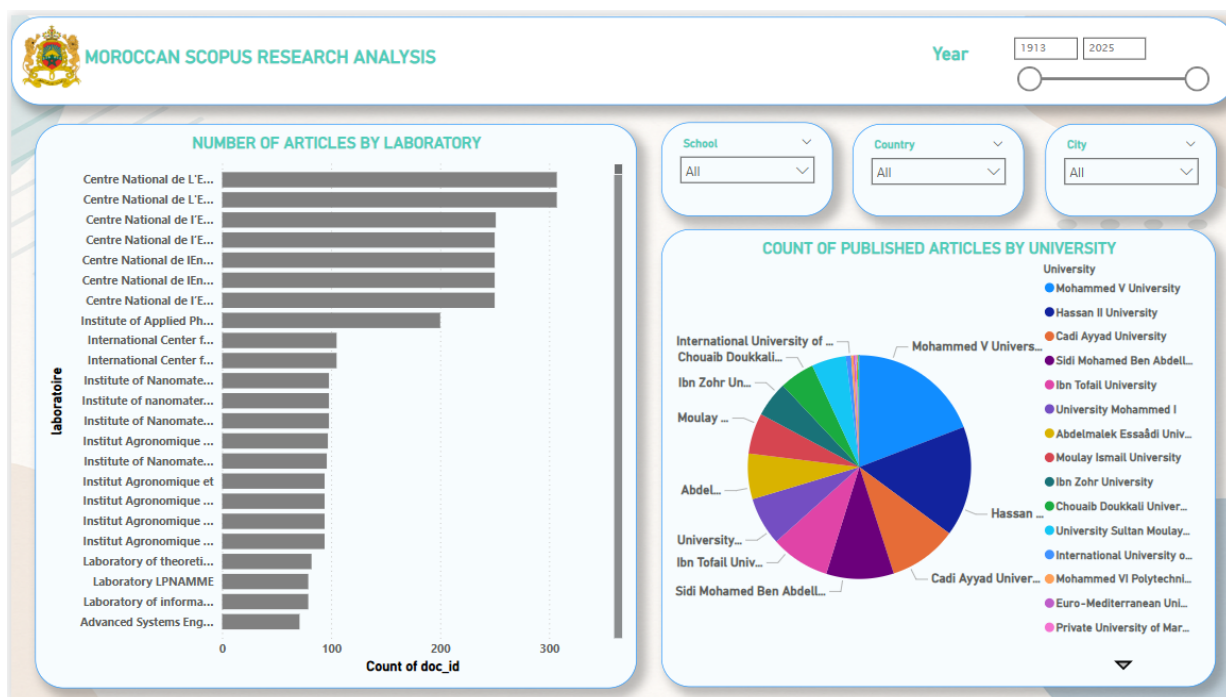


Figure 21 : Répartition des articles par type de publication et évolution temporelle (1913–2025)

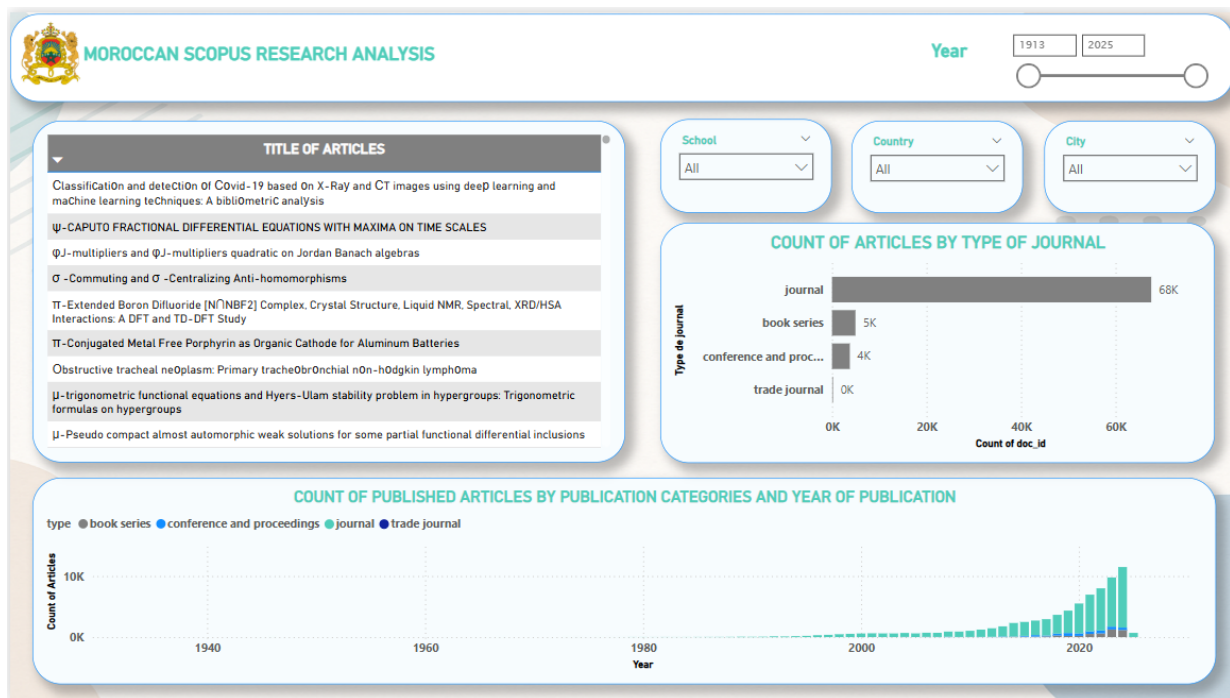
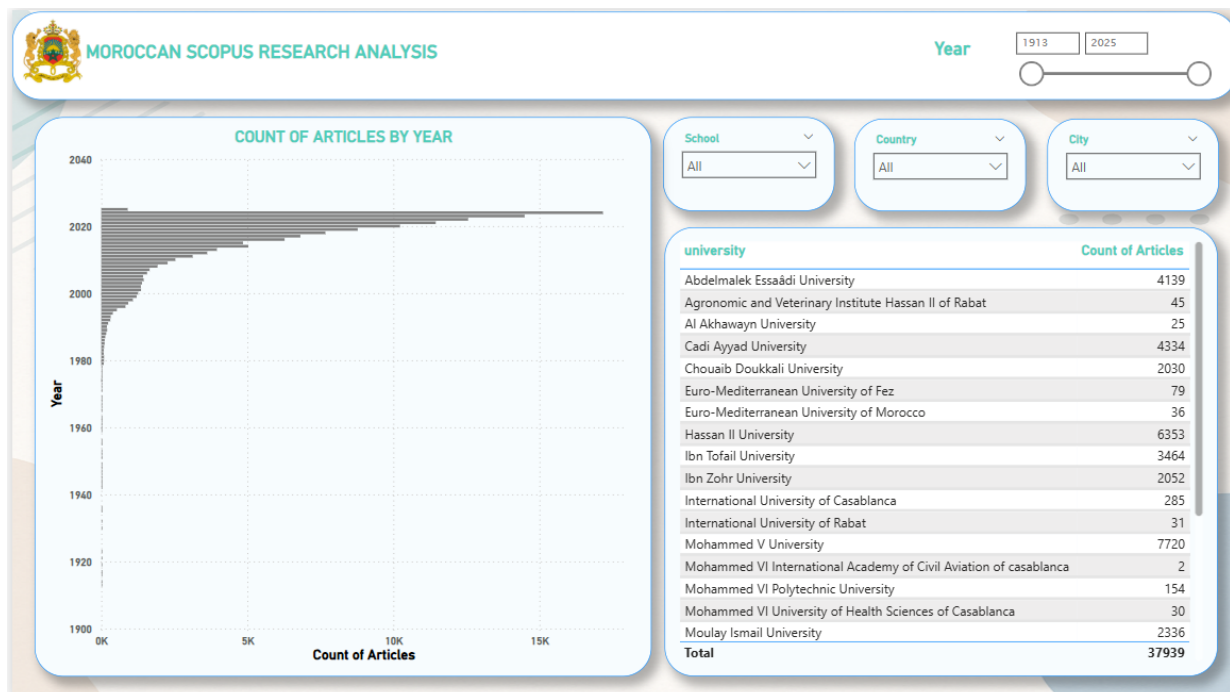


Figure 22 : Nombre d'articles par année et tableau comparatif par université



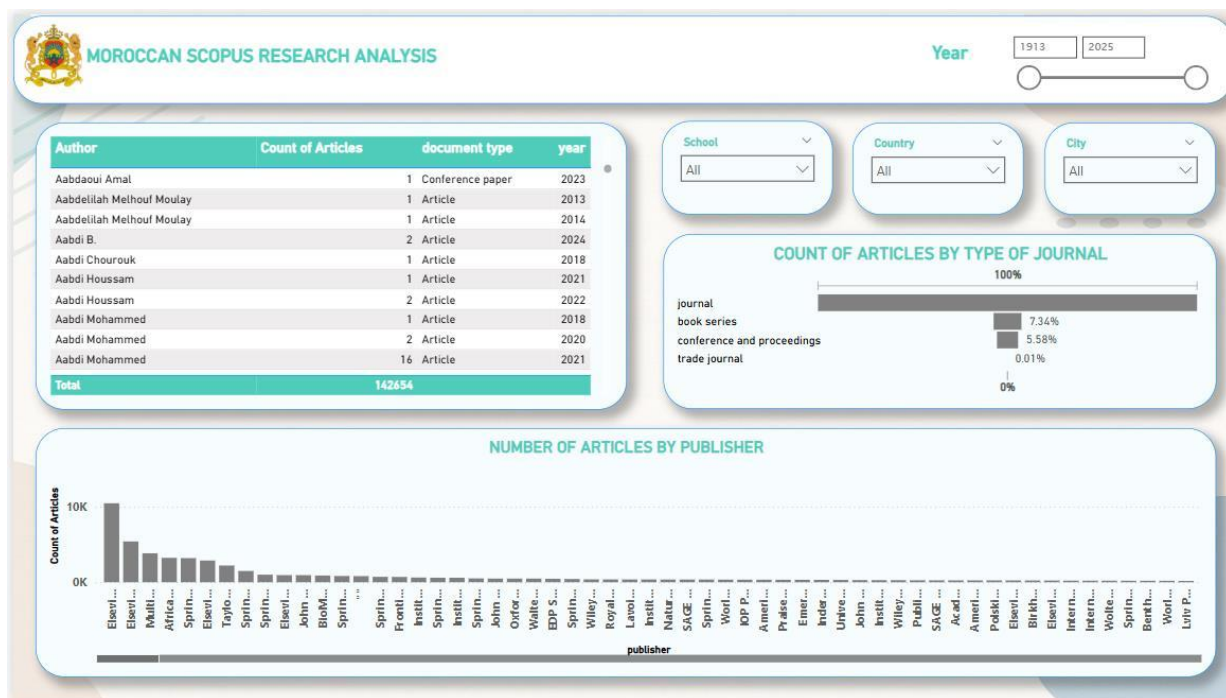


Figure 23 : Analyse par auteurs, types de journaux et éditeurs (142 654 auteurs total)

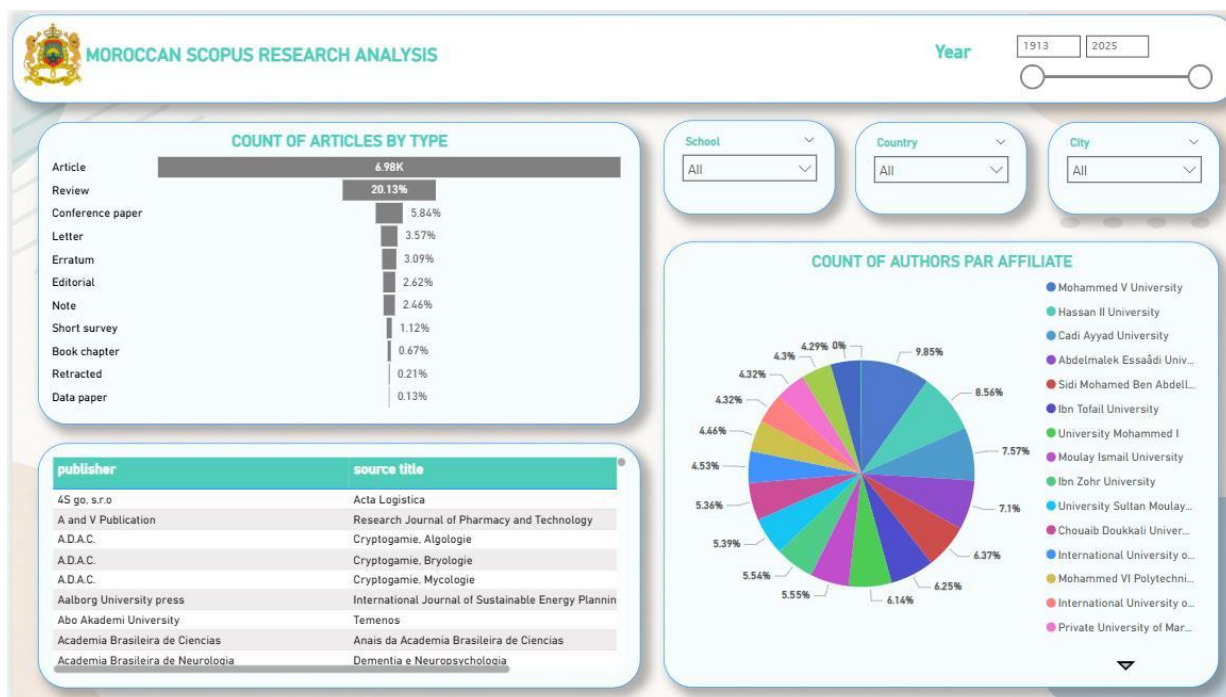


Figure 24 : Typologie des documents publiés et affiliations des auteurs par université

2. Analyse des résultats

2.1. Bilan des performances

Le projet a atteint l'ensemble de ses objectifs. L'architecture Data Lake sur AWS s'est révélée solide, flexible et performante :

~138 K Articles indexés (total)	~235 K Auteurs identifiés	~39 K Affiliations normalisées
---	-------------------------------------	--

- Amazon S3 : centralisation sans contrainte de volume ni de format, durabilité garantie.
- AWS Glue : catalogage automatique et transformations PySpark fiables sur volumes importants.
- Amazon Athena : requêtes SQL serverless rapides directement sur S3 (temps de scan typique < 1 s pour des requêtes de comptage).
- Power BI : visualisations dynamiques et filtres contextuels accessibles aux décideurs académiques.

2.2. Problèmes rencontrés et solutions

Problème	Solution apportée
Doublons dans affiliations et structures	Intégration d'étapes de nettoyage (dropDuplicates) dans le pipeline ETL.
Incohérence des noms d'universités/écoles (variantes, abréviations, fautes)	Fichiers JSON de correspondance (university.json, school.json, city.json) avec variantes standardisées.
Crawlers Glue ne détectant pas correctement les schémas	Affinement des paramètres de détection, planification cohérente des exécutions.
Erreurs de colonnes ambiguës dans Athena	Restructuration des requêtes SQL, sélection explicite des colonnes, jointures précisément définies.

Conclusion Générale et Perspectives

Ce projet de fin d'études a abouti à la conception et la mise en œuvre réussie d'une architecture Data Lake complète sur AWS pour le traitement et l'analyse des données bibliographiques Scopus des universités marocaines.

Sur le plan technique, l'approche serverless d'AWS a simplifié le déploiement en permettant de se concentrer sur la logique métier. Le pipeline automatisé (Lambda → Glue → Athena → Power BI) traite les données de l'ingestion brute jusqu'à la visualisation décisionnelle, en garantissant qualité, cohérence et traçabilité à chaque étape. La structuration Bronze/Silver/Gold s'est révélée particulièrement efficace pour la séparation des responsabilités et la réutilisabilité.

Sur le plan analytique, les tableaux de bord Power BI offrent une vision multidimensionnelle de la production scientifique marocaine : géographique, institutionnelle, temporelle, linguistique et thématique. Les résultats révèlent une prédominance de Mohammed V University (27 365 articles), une domination de l'anglais (90,58 %) et une accélération notable des publications à partir des années 2000.

D'un point de vue personnel, ce projet a renforcé notre maîtrise des technologies cloud, notre sens de l'organisation, notre capacité à résoudre des problèmes techniques concrets et notre esprit critique face aux choix d'implémentation. Il constitue un premier pas solide vers le monde professionnel du Data Engineering.

Les perspectives d'évolution incluent : l'intégration de Machine Learning (SageMaker), la gouvernance avancée (Lake Formation), l'orchestration renforcée (Step Functions), et l'extension à d'autres sources bibliographiques internationales. Ce travail pose une base robuste pour valoriser la donnée scientifique comme actif stratégique.

Bibliographie et Webliographie

- [1] J. Gantz and D. Reinsel, "The Digital Universe Driving Data Growth in Healthcare," IDC iView, 2014.
- [2] R. Kitchin, The Data Revolution: Big Data, Open Data, Data Infrastructures. SAGE Publications, 2014.
- [3] H. Fang, "Managing Data Lakes in Big Data Era," Proc. Int. Conf. Cyberworlds (CW), 2015, pp. 66–71.
- [4] Amazon Web Services, "Data Lakes and Analytics on AWS." <https://aws.amazon.com/big-data/datalakes-and-analytics/>
- [5] A. Gandomi and M. Haider, "Beyond the Hype: Big Data Concepts, Methods, and Analytics," Int. J. Inf. Manage., vol. 35, no. 2, pp. 137–144, 2015.
- [6] Amazon Web Services, "Amazon S3 Pricing." <https://aws.amazon.com/s3/pricing/>
- [7] Netflix Technology Blog, "Evolution of the Netflix Data Platform," 2016. <https://netflixtechblog.com>
- [8] T. White, Hadoop: The Definitive Guide, 4th ed. O'Reilly Media, 2015.
- [9] Amazon Web Services, "AWS Glue — Serverless Data Integration." <https://aws.amazon.com/glue/>
- [10] Amazon Web Services, "AWS Glue Spark and PySpark Jobs." https://docs.aws.amazon.com/glue/latest/dg/spark_and_pyspark.html
- [11] Amazon Web Services, "Overview of Workflows in AWS Glue." https://docs.aws.amazon.com/glue/latest/dg/workflows_overview.html
- [12] Amazon Web Services, "Amazon Athena — Interactive SQL on Amazon S3." <https://aws.amazon.com/athena/>
- [13] Microsoft, "Power Query Amazon Athena Connector." <https://learn.microsoft.com/en-us/power-query/connectors/amazon-athena>
- [14] Elsevier, "Scopus Content." <https://www.elsevier.com/products/scopus/content>
- [15] Elsevier, "Scopus Author Profiles." <https://www.elsevier.com/products/scopus/author-profiles>
- [16] Elsevier, "Scopus Metrics." <https://www.elsevier.com/products/scopus/metrics>